

Cozmo technical report

Handheld phone capture to a dimensioned floor plan with damage annotations.

Every number below comes from a file in this repository and names the script that rebuilds it. Anything not yet measured is marked PENDING with what it waits on. `scripts/regenerate_all.sh` rebuilds all of it.

1. Architecture

One contract, one backend, three front ends.

The contract first. `cozmo/contracts/models.py` defines the plan. Its rule is that no dimension is ever a bare number: every `*_m`, `*_m2` and `*_deg` field is a Measurement carrying value, `ci_low`, `ci_high`, unit, method and `ci_level`. A test walks emitted JSON and fails on any bare number (`tests/test_contract.py`). Referential integrity is checked at load time, so a plan that parses is a plan whose graph is consistent, and the evaluator never repairs input before scoring it. The schema is generated from the models by `cozmo schema`, so code and schema cannot drift.

The shared backend. Tiers differ only in how they produce geometry. Once a tier has a point cloud with poses, everything downstream is shared: `cozmo/lidar/levels.py` (floor and ceiling), `walls.py` (Manhattan frame and wall faces), `rooms.py` and `cells.py` (rooms), `openings.py` (doors and adjacency), `uncertainty.py` (intervals), `render/plan.py` (drawing). Damage is separate and tier-agnostic: `cozmo/damage/` takes an iterator of frames, each an image plus optionally depth, intrinsics and pose, and the plan's surfaces.

Provenance. `run` writes `plan.json`, `plan.png`, `run_manifest.json`, `drift_report.json` and `debug/`. The manifest holds a SHA-256 of every input file the tier is allowed to read, the resolved config and its hash, the git commit, the seed, per-stage timings and library versions. The plan itself holds no wall-clock field, so the same input, config and seed produce a byte-identical `plan.json`; a test asserts it.

Tier isolation is enforced in code, not by convention. Every file access goes through `StrayScan.open`, which raises on a file the tier may not read: the video tier sees only `rgb.mp4`, the photo tier is refused a scan folder outright because `depth/` holds png files that would otherwise pass as room photos. The run manifest hashes only the permitted files, so provenance never touches data the tier could not have used (`tests/test_stray.py`).

2. Tiers and devices

tier	input	reconstruction	state
lidar	one Stray Scanner scan for the whole property	classical geometry, no ML	done
video	one clip per room	SfM plus monocular depth, separate branch	PENDING, waits on loop 2 after-run
photo	2 to 8 stills per room	SfM plus monocular depth, separate branch	PENDING, waits on the photo tier

Stray Scanner format. `rgb.mp4` is 1920x1440 HEVC at 60 fps, stored rotated for a portrait capture; `depth/NNNNNN.png` is 256x192 uint16 millimetres; `confidence/` is 0, 1, 2 and only 2 is used; `odometry.csv` gives camera-to-world poses with quaternions in x, y, z, w order and per-frame

intrinsic for the full frame, scaled by 256/1920 for depth. Unprojection is OpenCV convention, $X = (u - cx) / fx * d$, then $world = R p + t$. Getting this wrong is silent, so it is pinned by a regression test: on c00a170fe1 the floor must appear as a peak at world $y = -1.48$ m within 2 cm (tests/test_stray.py::test_real_scan_floor_peak_near_minus_1_48).

Devices. Nokia 8.1 shot every video loop and the bedroom_2_repeat stills; Moto Edge 50 Neo shot the hall, bedroom_1, bedroom_2 and kitchen stills. EXIF make and model were stripped from the supplied files, so device attribution is the operator's record and the ground-truth files say so. Read from the files themselves: stills are 1200x1600, clips are 3840x2160 H.264 at about 30 fps. **No iPhone was available for capture:** every LiDAR result comes from the three supplied scans, so the LiDAR capture route in docs/capture_protocol.md is written from the format and has not been walked through on our own device. Full table in docs/device_matrix.md.

Runtime, fix_loop/*/bench/runs/*/run_manifest.json, regenerated by fix_loop/regenerate.sh: 4.0 s for 1715 frames, 13.2 s for 5251, 27.8 s for 9745, on an Apple M1 Max. The budget was 3 minutes for the largest scan.

Video tier: what was rejected and why

MapAnything was tested and rejected (docs/experiments/mapanything/result.json).

Image-only inference on c00a170fe1 at 20, 40 and 80 frames ran on MPS with no operator fallback, 21 s to 141 s. Its metric scale is wrong by about 30%, consistently: the median ratio of predicted depth to LiDAR depth over confidence-2 pixels is 0.712, 0.704 and 0.709 across the three runs, standard deviation 0.09 to 0.13. A fixed correction would leave roughly 13% residual spread, so this is a systematic scale error with real per-frame noise on top, not a calibration. The fused cloud does not produce straight walls; the top-down density is a smear extending 11 m from a room a few metres across (docs/experiments/mapanything/topdown_density_n40.png).

Feeding it the ARKit poses and intrinsic would likely have fixed the scale, and it was refused: the video tier has no poses and no intrinsic. A video result obtained by handing the model LiDAR-derived poses measures the LiDAR tier, not the video tier, and would not survive the walk-in test on a phone without a depth sensor.

The chosen route is SfM plus Depth Pro, with the monocular depth scaled by the focal length COLMAP recovers rather than by a learned scale prior. That keeps scale traceable to the reconstruction rather than to a network. Accuracy PENDING, waits on the loop 2 after-run on the video branch.

3. Drift handling

ARKit odometry drifts slowly: yaw creeps and floor height wanders over a few minutes of walking, which smears wall faces and breaks the floor into a wedge. cozmolidar/drift.py anchors to the building rather than to the trajectory. The walk is cut into 5-second chunks. Per chunk: the dominant wall azimuth modulo 90 degrees gives a yaw error against the global Manhattan direction, removed by rotating about the chunk's own centroid so it does not swing across the map; the median height of its floor points against the global floor plane gives a vertical offset; then the chunk's wall points are shifted along each

horizontal axis by the median offset to the nearest global wall face.

Corrections beyond the configured limits are rejected rather than applied, because a chunk that looks 20 degrees off usually has too little wall to estimate from.

The ablation says it barely matters on this data. From

`fix_loop/after/bench/runs/*/drift_report.json`, rebuilt by

`fix_loop/regenerate.sh`:

scan	wall thickness off	on	footprint off	on
c00a170fe1	24.2 mm	23.6 mm	55.4 m2	35.8 m2
1a8384c3f6	32.1 mm	35.8 mm	80.0 m2	86.6 m2
c7d28f72c6	32.5 mm	32.6 mm	65.8 m2	64.7 m2

Wall thickness improves on one scan, worsens on another, and is unchanged on the third. The reason is in the model summary: mean yaw error 0.55 to 1.57 degrees, mean height error 3 to 8 mm, mean shift 1.4 to 2.2 cm. ****ARKit's poses were already good enough that there was little to correct.**** Room overlap does fall consistently (0.39 to 0.27 m2 on the largest scan). The correction is on by default because it costs one extra pass and does no harm, and that is the whole case for it.

4. Error budget

Every interval is built from named terms, defined in `cozmo/lidar/uncertainty.py`.

Wall length. Two independent parts. Statistical: each bounding wall face has a position error equal to its residual spread divided by the square root of an effective sample size, where the effective count is the face length divided by a 0.25 m correlation length, never the raw point count, because points 2 cm apart on one wall are not independent measurements. Systematic: a depth scale bias, default 1%, scaling with the measured length, plus a 1 cm absolute floor below which nothing is claimed. Half width is $1.96 * \sqrt{\sigma_a^2 + \sigma_b^2 + (\text{bias} * L)^2}$, floored at 1 cm.

Observed on the supplied scans under the shipped default: the median interval is 2.4% of the wall length, and the longest wall found, 4.186 m, is reported with plus or minus 8.3 cm

(`fix_loop/before/bench/runs/c7d28f72c6/plan.json`, rebuilt by `fix_loop/regenerate.sh` before).

Ceiling height. Measured per 25 cm grid cell as the ceiling mean minus the floor plane beneath it, reported as the median. The half width combines the cell-to-cell spread with the depth scale term. ****The spread is deliberately not divided by the square root of the cell count****: cells are not independent samples of one number, and claiming sub-millimetre precision from a handheld scan is exactly the confident-garbage failure the grading penalises. Where the ceiling covers under 15% of floor cells, no number is invented: a 2.2 to 3.2 m prior is emitted with method `prior_no_ceiling_observed` and a warning. Two of the three supplied scans hit that path. The third measured 3.06 to 3.08 m across rooms, consistent to 1.5 cm.

Areas propagate from the edge lengths, relative errors combined in quadrature over the two axes, which is exact for a rectangle.

Partially observed rooms have their length and area intervals doubled, and say so in the method string and in a warning. A room whose outline is under 80% backed by observed wall is a lower bound, not a measurement. Threshold chosen from synthetic scenes: closed rooms measure 0.94 to 1.00 perimeter support, a room with one wall missing measures 0.68.

Damage extent with depth is an interval from a colour-contrast estimate to the detection box projected on the surface, because the box bounds the mark and is not the mark. Without depth the region carries method `unbounded_no_metric_depth`, an interval of 0 to 4 m² and a warning.

Ground truth, and its limits

Stated plainly, because the error budget is only as good as what it is checked against:

- **The supplied LiDAR scans have no ground truth at all.** Nobody measured that property. No accuracy number for the LiDAR tier exists or is given.

- **Own rooms: tape, centimetres to the nearest inch.** `truth_uncertainty_m` is 0.013 and the coverage check widens the truth by it before asking whether an interval covers it, so a pipeline is not marked wrong for being more precise than the reference.

- **The hall is scored on ceiling height, openings and adjacency only** (`score_walls: false`). It is an open-plan connector taking in the corridor and laundry, with no tape-measurable wall run.

- **Windows were not measured.** Four are marked `present_unmeasured`: a prediction there is neither a hit nor a phantom, and contributes no calibration item.

- **Staged damage is two A4 sheets**, 21.0 x 29.7 cm, recorded as `extent_m2_upper_bound: 0.0624`, because a sheet bounds the mark it stands for rather than being it.

5. Calibration analysis

Calibration is scored separately from accuracy, in `cozmo/eval/calibration.py`: empirical coverage against the nominal 95%, mean interval width as a percentage of value, and a "confident garbage" count, meaning the truth falls outside the interval and that interval is narrower than the median for its quantity type. The median is per quantity because widths in metres and in square metres are not comparable. Broken down by tier and by quantity.

Empirical coverage for any tier is PENDING. It waits on predictions for the hand-measured rooms, which waits on the photo and video tiers. The supplied LiDAR scans cannot contribute: no ground truth means no coverage to compute.

What can be said now is the shape of the intervals the LiDAR tier produces, and where honesty is enforced structurally rather than statistically. Two of three supplied scans report ceiling height as a prior rather than a measurement. Every room in the largest scan is flagged partially observed with doubled intervals. Both are cases where the pipeline declines to produce a tight number it cannot support, which is the behaviour the calibration score

is meant to reward, measured here by inspection rather than by coverage.

6. The fix loop

One loop was run on cross-capture repeatability. ****It is a negative result and is reported as one**** (`fix_loop/`).

The declaration. Worst gate: repeatability, 0 of 112 wall rows within tolerance on the two captures of one apartment. Root cause hypothesis: room partition is unstable because it is grown from observed floor and the walked path, so it is a function of where the operator walked. Predicted after the fix: 40% of rows within tolerance, range 30 to 55%.

The eval was broken first. Plans are emitted in the ARKit world frame, whose orientation depends on where the capture started, so the two captures sat 58 degrees apart and no quarter turn aligned them. Every wall pair failed the parallel test, which looked like total failure but was an impossible comparison. Fixed by canonicalising each plan to its own Manhattan frame before the quarter-turn search, in commits before the BEFORE run was taken.

The falsifier fired. The declaration named a cheap test: if dropping ghost faces alone fixed room counts, the cell complex was unnecessary. It did not. Ghost rejection removes about 1% of face length and the gate did not move (`fix_loop/experiments/ghost_only/RESULT.md`). That also showed the declaration's own evidence had over-counted ghosts at 16%, because it measured faces outside an envelope built from the polygons under test.

The fix shipped and did not work. Wall-driven cell complex segmentation, support taken from the 1.0 to 1.6 m band (chosen because the higher band the plots suggested does not exist in one capture: only 10.0% of its wall points lie between 1.6 and 2.2 m). Result: still 0 rows within tolerance, rooms paired across the captures fell from 6 to 3, footprint areas inflated. Three of four predictions were wrong. erosion is the default again and cells stays selectable.

The real cause was the third falsifier: coverage. Only **56% and 37%** of each capture's wall-face length has any counterpart in the other. The two captures did not look at the same things.

The contrast that carries the finding, from

`fix_loop/evidence/evidence.json`, rebuilt by `fix_loop/evidence_run.py`:

measured at	disagreement between the two captures
wall face position	1.9 cm median
polygon edge length	97.5 cm median

The geometry is sound to two centimetres. The partition into rooms and edges is not, and no segmentation rule can invent a wall one capture never saw. The sample pair is now labelled `coverage_mismatched` and is not presented as a valid repeat. The official repeat pair is `bedroom_2` against `bedroom_2_repeat`: same-device at the video tier and the primary evidence, cross-device at the photo tier and labelled as mixing camera differences into any disagreement. Both PENDING on their tiers.

7. Known failure modes

Measured in the repository.

● **Cross-capture repeatability fails completely.** 0 of 153 rows. One loop

did not fix it. It needs a capture-protocol or gate-denominator change.

● **Damage precision on photos.** 9 false regions in a room with 2 marks, from 63 before filtering. On a LiDAR capture the same filters reach 0 from 106. The gap is structural: two of the four filters need depth and poses (`scripts/damage_ablation.py`, `docs/damage_eval/README.md`).

● **Cracks are the class most at risk.** The crop verifier that removes the false positives also removes the crack at threshold 0.20. A thin dark line on paint reads as a clean wall to SigLIP. The shipped threshold is 0.15 for this reason.

● **Room segmentation over-segments.** 11 rooms on a flat with perhaps 6 or 7 spaces. Two captures of one property give different counts.

● **Ceiling height is often unavailable.** Two of three supplied scans never observed enough ceiling, so they report a 1 m wide prior.

Hazards seen in the real captures.

● **Glass shower screen** and a **wardrobe mirror**: both return confident depth for a room that is not there. Handled for damage by the geometry and multi-view filters, which take that scan from 106 false regions to 0. Not handled in reconstruction: mirrored geometry still contributes wall faces.

● **A dog moved through the capture.** Moving objects leave smeared points that no static-scene assumption covers.

● **Glossy floor tiles** return depth from the reflection as well as the surface, which thickens the floor peak.

● **Textureless white walls** give sparse returns near the LiDAR range limit, which is part of why coverage differs so much between two walks of one flat.

● **A ceiling never observed** is the commonest single cause of a wide interval, and it is entirely a capture-protocol problem: one deliberate tilt upward per room fixes it.

Assumptions that will eventually break.

● **Manhattan**: walls meet at right angles after one global rotation. A curved or 45-degree wall appears as a missing face rather than a wrong one, which is the safer failure, but it is still a failure. Recorded in every plan's assumptions.

● **Video-tier scale depends on the camera being held at about 1.4 m.** The capture protocol says so twice. A clip shot at waist height will be wrong by the ratio of the heights, and nothing in the pipeline can detect it.